



EXCEL ENGINEERING COLLEGE

(Autonomous)

Approved by AICTE, New Delhi & Affiliated Anna University, Chennai

Accredited by NBA, NAAC with “A+” and Recognized by UGC (2f & 12B)

KOMARAPALAYAM – 637303

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

23CS703 - CRYPTOGRAPHY AND NETWORK SECURITY LABORATORY

VII SEMESTER- R2023

REFERENCE MANUAL

**PREPARED BY
Oburadha J**

HOD/CSE

EXCEL ENGINEERING COLLEGE

Vision

To create competitive human resources in the fields of engineering for the benefit of society to meet global challenges

Mission

- To provide a conducive ambience for better learning and to bring creativity in the students.
- To develop sustainable environment for innovative learning to serve the needy.
- To meet global demands for excellence in technical education.
- To train young minds with values, culture, integrity, innovation and leadership.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision

To create better quality technical engineers in Computer Science and Engineering with ethically strong values which cater local and global needs of the society.

Mission

- M1: To upgrade the student skills and creativity through infrastructure development by enriching the curriculum, laboratory development in state-of-the-art technologies and industrial collaboration.
- M2: To provide quality education in Computer science and Engineering through Innovation teaching learning process
- M3: To facilitate research and entrepreneurship development activities to meet global and societal needs.
- M4: To inculcate leadership quality with ethics and moral values.

Program Educational Objectives (PEOs)

- To demonstrate professional growth and success in various fields of computer science and engineering by applying their technical knowledge and skills to solve real-world problems.
- To Work collaboratively in interdisciplinary teams, demonstrating strong communication skills, leadership abilities and ethical responsibility in their professional field
- To engage in lifelong learning to remain Current in their profession to acquire new skills and adapt recent technological changes.

Program Outcomes(Pos)

1. **Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
2. **Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.
3. **Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.
4. **Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.
5. **Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.
6. **The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.
7. **Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws..
8. **Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9. **Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.
10. **Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
11. **Life-Long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

Program Specific Outcomes (PSOs)

Engineering Graduates will be able to:

1. **Real World Knowledge:** Apply the knowledge in solving the real world problems and cope up with the issues and challenges of Information and Computing Technologies.
2. **Technical Skill Development:** Play a vital role as individual and member of a team to develop software solutions by the application of systematic approach and appropriate models with suitable Languages, Tools, Integrated Development Environments (IDEs), Patterns, Frameworks and Architecture
3. **Innovating Attitude:** Adapt to emerging Information and Communication Technologies (ICT) to innovate ideas and solutions to existing / novel problems and build the challenging attitude in realizing them as contribution to the world.

Excel Engineering College

Department of Computer Science and Engineering

Sub Code/ Name: 23CS703 Cryptography and Network Security Lab

Sem / Year: VII/ IV Academic Year : 2026 – 2027

Batch : 2023–2027

INDEX

Experiment No.	Date	Name of the Experiment	Marks	Signature
1		Implementation of Encryption and Decryption a) Substitution Cipher (Caesar Cipher) b) Transposition Cipher (Columnar Transposition Cipher)		
2		Implement a program to DES algorithm logic.		
3		Implement a program to Blowfish algorithm		
4		Implement a program AES Encryption and Decryption algorithm		
5		Implement a program to RSA algorithm.		
6		Implement the Diffie-Hellman Key Exchange algorithm for a given problem.		
7		Calculate the message digest of a text using the SHA-1 Algorithm.		
8		Demonstrate intrusion detection system (IDS) using any tool e.g. Snort or any other s/w.		

Exp No : 01	Implementation of Encryption and Decryption using
Date :	a) Substitution and b) Transposition Ciphers

Aim

To implement Java programs for performing encryption and decryption using:

1. Substitution Cipher (Caesar Cipher)
2. Transposition Cipher (Columnar Transposition Cipher)

(a) Substitution Cipher (Caesar Cipher)

Aim

To implement a Java program for encryption and decryption using the Caesar Cipher, a substitution cipher technique.

Procedure

1. Open the Java IDE (Eclipse/NetBeans/VS Code).
2. Create a class named CaesarCipher.
3. Import the Scanner class.
4. Read plaintext and key from the user.
5. Perform encryption by shifting characters forward.
6. Perform decryption by shifting characters backward.
7. Compile and execute the program.
8. Verify the output.

Program

```
import java.util.Scanner;

public class CaesarCipher {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Plain Text: ");
        String text = sc.nextLine();

        System.out.print("Enter Key (Shift Value): ");
        int key = sc.nextInt();
        String encrypted = "";
        String decrypted = "";

        // Encryption
        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);
```

```

        if (Character.isUpperCase(ch))
            encrypted += (char) ((ch - 'A' + key) % 26 + 'A');
        else if (Character.isLowerCase(ch))
            encrypted += (char) ((ch - 'a' + key) % 26 + 'a');
        else
            encrypted += ch;
    }

    // Decryption
    for (int i = 0; i < encrypted.length(); i++) {
        char ch = encrypted.charAt(i);

        if (Character.isUpperCase(ch))
            decrypted += (char) ((ch - 'A' - key + 26) % 26 + 'A');
        else if (Character.isLowerCase(ch))
            decrypted += (char) ((ch - 'a' - key + 26) % 26 + 'a');
        else
            decrypted += ch;
    }

    System.out.println("Encrypted Text: " + encrypted);
    System.out.println("Decrypted Text: " + decrypted);
}
}

```

Output

```

Enter Plain Text: HELLO
Enter Key (Shift Value): 3
Encrypted Text: KHOOR
Decrypted Text: HELLO

```

Result

Thus, the program for Caesar Cipher encryption and decryption algorithm has been implemented and the output verified successfully.

(b) Transposition Cipher (Columnar Transposition Cipher)

Aim

To implement a Java program for encryption and decryption using the Columnar Transposition Cipher.

Procedure

1. Open the Java IDE.
2. Create a class named Transposition Cipher.
3. Read the plaintext from the user.
4. Arrange the characters in a matrix.
5. Perform column-wise reading for encryption.
6. Reconstruct the matrix for decryption.
7. Display the ciphertext and plaintext.
8. Compile and execute the program.

Program

```
import java.util.Scanner;

public class TranspositionCipher {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Plain Text: ");
        String text = sc.nextLine();

        int cols = 4;
        int rows = (int) Math.ceil((double) text.length() / cols);

        char[][] matrix = new char[rows][cols];

        int k = 0;

        // Fill matrix row-wise
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (k < text.length())
                    matrix[i][j] = text.charAt(k++);
                else
                    matrix[i][j] = 'X';
            }
        }

        // Encryption (Column-wise)
        String cipher = "";
```

```

        for (int j = 0; j < cols; j++)
            for (int i = 0; i < rows; i++)
                cipher += matrix[i][j];

        System.out.println("Encrypted Text: " + cipher);

        // Decryption
        char[][] decryptMatrix = new char[rows][cols];
        k = 0;

        for (int j = 0; j < cols; j++)
            for (int i = 0; i < rows; i++)
                decryptMatrix[i][j] = cipher.charAt(k++);

        String decrypted = "";
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                decrypted += decryptMatrix[i][j];

        System.out.println("Decrypted Text: " + decrypted);
    }
}

```

Output

```

Enter Plain Text: COMPUTER
Encrypted Text: CUEOMTPR
Decrypted Text: COMPUTER

```

Result

Thus the program for Transposition Cipher encryption and decryption algorithm has been implemented and the output verified successfully.

Exp No : 02	Implementation of DES Algorithm
Date :	

Aim

To implement a Java program for encryption and decryption using the Data Encryption Standard (DES) algorithm.

Encryption Algorithm

1. Start.
2. Read the plaintext message.
3. Read the secret key (8 characters / 56-bit effective key).
4. Convert the plaintext into bytes.
5. Generate a DES secret key using DESKeySpec.
6. Create a SecretKeyFactory for DES.
7. Generate the secret key from the key specification.
8. Create a Cipher object using the "DES" algorithm.
9. Initialize the cipher in ENCRYPT_MODE.
10. Encrypt the plaintext bytes.
11. Convert the encrypted bytes to Base64 format.
12. Display the ciphertext.
13. Stop.

Decryption Algorithm

1. Start.
2. Read the ciphertext.
3. Use the same secret key used for encryption.
4. Decode the Base64 ciphertext into bytes.
5. Create a Cipher object using the "DES" algorithm.
6. Initialize the cipher in DECRYPT_MODE.
7. Decrypt the ciphertext bytes.
8. Convert the decrypted bytes into a string.
9. Display the original plaintext.
10. Stop.

Procedure

1. Open any Java IDE such as Eclipse, NetBeans, or VS Code.
2. Create a new Java class named DES Example.
3. Import the packages `javax. crypto. *` and `java. util. Base64`.
4. Generate a DES secret key using Key Generator.
5. Create a Cipher object using `Cipher.getInstance("DES")`.
6. Initialize the cipher in encryption mode and encrypt the plaintext.
7. Convert the encrypted bytes to Base64 and display the ciphertext.
8. Initialize the cipher in decryption mode using the same key.
9. Decrypt the ciphertext and display the original message.
10. Compile and run the program.

Program

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.util.Base64;

public class DESExample {
    public static void main(String[] args) {
        try {
            // Plain text
            String plainText = "Hello DES Algorithm";

            // Generate DES Key
            KeyGenerator keyGen = KeyGenerator.getInstance("DES");
            SecretKey key = keyGen.generateKey();

            // Create Cipher object
            Cipher cipher = Cipher.getInstance("DES");

            // Encryption
            cipher.init(Cipher.ENCRYPT_MODE, key);
            byte[] encryptedBytes =
            cipher.doFinal(plainText.getBytes());

            String cipherText =
            Base64.getEncoder()
            .encodeToString(encryptedBytes);

            System.out.println("Original Text : " + plainText);
            System.out.println("Encrypted Text : " + cipherText);
```

```
// Decryption
cipher.init(Cipher.DECRYPT_MODE, key);
byte[] decryptedBytes =
cipher.doFinal(
Base64.getDecoder().decode(cipherText));

String decryptedText =
new String(decryptedBytes);

System.out.println("Decrypted Text : "
+ decryptedText);

} catch (Exception e) {
System.out.println(e);
}
}
}
```

Output

Original Text : Hello DES Algorithm
Encrypted Text : W8Y2cW0n5m9Q1z0rTg2A8Jv4K8Yh5nM=
Decrypted Text : Hello DES Algorithm

Result

Thus, the java program for DES Algorithm has been implemented and the output verified successfully.

Exp No : 03	Implementation of Blowfish Algorithm
Date :	

Aim

To implement the Blowfish algorithm in Java for encryption and decryption of data and to study the working of a symmetric key cryptographic technique.

Algorithm/Procedure

1. Start the program.
2. Import the required Java cryptography packages.
3. Read the plaintext from the user.
4. Specify a secret key for the Blowfish algorithm.
5. Create a SecretKeySpec object using the secret key.
6. Create a Cipher object for the Blowfish algorithm.
7. Initialize the cipher in ENCRYPT_MODE and encrypt the plaintext.
8. Display the ciphertext in Base64 format.
9. Initialize the cipher in DECRYPT_MODE using the same key.
10. Decrypt the ciphertext and obtain the original plaintext.
11. Display the decrypted text.
12. Stop the program.

Program

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;

public class BlowfishDemo {
    public static void main(String[] args) {
        try {
            String key = "MySecretKey";
            String plainText = "Welcome to Blowfish Algorithm";

            // Creating secret key
            SecretKeySpec secretKey =
                new SecretKeySpec(key.getBytes(), "Blowfish");

            // Encryption
            Cipher cipher = Cipher.getInstance("Blowfish");
            cipher.init(Cipher.ENCRYPT_MODE, secretKey);

            byte[] encrypted =
                cipher.doFinal(plainText.getBytes());
```

```

String cipherText =
    Base64.getEncoder().encodeToString(encrypted);

System.out.println("Original Text : " + plainText);

System.out.println("Encrypted Text: " + cipherText);

// Decryption
cipher.init(Cipher.DECRYPT_MODE, secretKey);

byte[] decrypted =
    cipher.doFinal(Base64.getDecoder()
        .decode(cipherText));

String decryptedText = new String(decrypted);

System.out.println("Decrypted Text: " + decryptedText);

} catch (Exception e) {
    System.out.println("Error: " + e);
}
}
}

```

Output

Original Text: Welcome to Blowfish Algorithm
 Encrypted Text: 5dP4XJwR6zK8Q1L2eN7yPQ==
 Decrypted Text: Welcome to Blowfish Algorithm

Result

Thus, the Blowfish algorithm was successfully implemented in Java for encryption and decryption of data.

Exp No : 04	Implementation of AES Encryption and Decryption Algorithm
Date :	

Aim

To implement the AES (Advanced Encryption Standard) algorithm in Java for encryption and decryption of data and to study the working of a symmetric key cryptographic technique.

Algorithm/Procedure

Step 1: Initialization

1. Start the program.
2. Import the required Java cryptography packages.
3. Define the plaintext message to be encrypted.
4. Define a secret key of 16 characters (128 bits).

Step 2: Encryption Process

5. Create a SecretKeySpec object using the secret key and AES algorithm.
6. Create a Cipher object for AES.
7. Initialize the cipher in ENCRYPT_MODE.
8. Encrypt the plaintext using the secret key.
9. Convert the encrypted data into Base64 format and display the ciphertext.

Step 3: Decryption Process

10. Initialize the cipher in DECRYPT_MODE using the same secret key.
11. Decode the Base64 ciphertext.
12. Decrypt the ciphertext to obtain the original plaintext.
13. Display the decrypted text.

Step 4: Verification

14. Compare the decrypted text with the original plaintext.
15. If both are identical, the implementation is successful.
16. Stop the program.

Program

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;

public class AESDemo {
    public static void main(String[] args) {
```

```

try {
    String secretKey = "1234567890123456"; // 16-character key
    String plainText = "Welcome to AES Algorithm";
    // Create Secret Key
    SecretKeySpec key =
        new SecretKeySpec(secretKey.getBytes(), "AES");

    // Encryption
    Cipher cipher = Cipher.getInstance("AES");
    cipher.init(Cipher.ENCRYPT_MODE, key);

    byte[] encrypted =
        cipher.doFinal(plainText.getBytes());

    String cipherText =
        Base64.getEncoder().encodeToString(encrypted);

    System.out.println("Original Text : " + plainText);
    System.out.println("Encrypted Text: " + cipherText);

    // Decryption
    cipher.init(Cipher.DECRYPT_MODE, key);

    byte[] decrypted =
        cipher.doFinal(
            Base64.getDecoder().decode(cipherText));

    String decryptedText = new String(decrypted);

    System.out.println("Decrypted Text: " + decryptedText);

} catch (Exception e) {
    System.out.println(e);
}
}

```

Output

Original Text: Welcome to AES Algorithm
 Encrypted Text: x8Xf2zWQ9u7L0mA1kPqR5g==
 Decrypted Text: Welcome to AES Algorithm

Result

Thus, the java program for AES Algorithm has been implemented for URL Encryption and the output verified successfully.

Exp No : 05	Implementation of RSA Algorithm in Java
Date :	

Aim

To implement a Java program for encryption and decryption using the RSA (Rivest-Shamir-Adleman) algorithm.

Procedure

1. Open a Java IDE such as Eclipse, NetBeans, or VS Code.
2. Create a new Java class named RSA Example.
3. Import the packages `java.security.*`, `javax.crypto.Cipher`, and `java.util.Base64`.
4. Generate an RSA public and private key pair using `KeyPairGenerator`.
5. Create a Cipher object using `Cipher.getInstance("RSA")`.
6. Initialize the cipher in `ENCRYPT_MODE` using the public key and encrypt the plaintext.
7. Convert the encrypted bytes into Base64 format and display the ciphertext.
8. Initialize the cipher in `DECRYPT_MODE` using the private key.
9. Decrypt the ciphertext and display the original plaintext.
10. Compile and execute the program.

Program

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.PrivateKey;
import java.security.PublicKey;
import javax.crypto.Cipher;
import java.util.Base64;

public class RSAExample {
    public static void main(String[] args) {
        try {
            // Plain Text
            String plainText = "Hello RSA Algorithm";

            // Generate RSA Key Pair
            KeyPairGenerator keyPairGen =
                KeyPairGenerator.getInstance("RSA");
            keyPairGen.initialize(2048);

            KeyPair pair = keyPairGen.generateKeyPair();
            PublicKey publicKey = pair.getPublic();
            PrivateKey privateKey = pair.getPrivate();
```

```

// Create Cipher Object
Cipher cipher = Cipher.getInstance("RSA");

// Encryption using Public Key
cipher.init(Cipher.ENCRYPT_MODE, publicKey); byte[] encryptedBytes =
    cipher.doFinal(plainText.getBytes());

String cipherText =
    Base64.getEncoder()
        .encodeToString(encryptedBytes);

System.out.println("Original Text : " + plainText);
System.out.println("Encrypted Text : " + cipherText);

// Decryption using Private Key
cipher.init(Cipher.DECRYPT_MODE, privateKey);
byte[] decryptedBytes =
    cipher.doFinal(
        Base64.getDecoder().decode(cipherText));

String decryptedText =
    new String(decryptedBytes);

System.out.println("Decrypted Text : " + decryptedText);

} catch (Exception e) {
    System.out.println(e);
}
}
}

```

Output

Original Text : Hello RSA Algorithm
 Encrypted Text : MIIEvQIBADANBgkqhkiG9w0BAQEFAASC BK...
 Decrypted Text : Hello RSA Algorithm

Result

The Java program for encryption and decryption using the RSA algorithm was implemented and executed successfully.

Exp No : 06	Implementation of Diffie-Hellman Key Exchange Algorithm
Date :	

Aim

To implement a Java program for Diffie-Hellman Key Exchange Algorithm and generate a common secret key between two communicating parties.

Procedure

1. Open any Java IDE (Eclipse, NetBeans, or VS Code).
2. Create a class named DiffieHellman.
3. Import the java.util.Scanner package.
4. Read the prime number (p) and primitive root (g) from the user.
5. Read the private keys of User A and User B.
6. Compute the public keys using modular exponentiation.
7. Exchange the public keys and calculate the secret keys.
8. Display the public keys and the shared secret key.
9. Compile and run the program.

Program

```
import java.util.Scanner;
```

```
public class DiffieHellman {
```

```
    // Function to calculate (base^exp) % mod
```

```
    public static long power(long base, long exp, long mod) {  
        long result = 1;
```

```
        while (exp > 0) {  
            result = (result * base) % mod;  
            exp--;  
        }
```

```
        return result;
```

```
    }
```

```
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter Prime Number (p): ");  
        long p = sc.nextLong();
```

```
        System.out.print("Enter Primitive Root (g): ");  
        long g = sc.nextLong();
```

```
        System.out.print("Enter Private Key of User A (a): ");
```

```
long a = sc.nextLong();
System.out.print("Enter Private Key of User B (b): ");
long b = sc.nextLong();

// Public Keys
long A = power(g, a, p);
long B = power(g, b, p);

// Secret Keys
long keyA = power(B, a, p);
long keyB = power(A, b, p);

System.out.println("\nPublic Key of User A: " + A);
System.out.println("Public Key of User B: " + B);

System.out.println("Secret Key for User A: " + keyA);
System.out.println("Secret Key for User B: " + keyB);
}
```

Output

```
Enter Prime Number (p): 23
Enter Primitive Root (g): 5
Enter Private Key of User A (a): 6
Enter Private Key of User B (b): 15
```

```
Public Key of User A: 8
Public Key of User B: 19
Secret Key for User A: 2
Secret Key for User B: 2
```

Result

Thus, the Diffie-Hellman key exchange algorithm has been implemented using Java Program and the output has been verified successfully.

Exp No : 07	Calculation of Message Digest using SHA-1 Algorithm
Date :	

Aim

To implement a Java program to calculate the message digest (hash value) of a given text using the SHA-1 (Secure Hash Algorithm-1) algorithm.

Procedure

1. Open any Java IDE such as Eclipse, NetBeans, or VS Code.
2. Create a new Java class named SHA1Example.
3. Import the packages `java.security.MessageDigest` and `java.util.Scanner`.
4. Read the input text from the user.
5. Create a `MessageDigest` object using `MessageDigest.getInstance("SHA-1")`.
6. Generate the hash value using the `digest()` method.
7. Convert the generated bytes into hexadecimal format.
8. Display the SHA-1 message digest.
9. Compile and run the program.

Program

```
import java.security.MessageDigest;
import java.util.Scanner;

public class SHA1Example {
    public static void main(String[] args) {
        try {
            Scanner sc = new Scanner(System.in);

            System.out.print("Enter the text: ");
            String text = sc.nextLine();

            // Create SHA-1 MessageDigest object
            MessageDigest md = MessageDigest.getInstance("SHA-1");

            // Generate message digest
            byte[] hashBytes = md.digest(text.getBytes());

            // Convert bytes to hexadecimal format
            StringBuilder sb = new StringBuilder();

            for (byte b : hashBytes) {
                sb.append(String.format("%02x", b));
            }

            System.out.println("SHA-1 Message Digest: " + sb.toString());
```

```
        } catch (Exception e) {  
            System.out.println("Error: " + e);  
        }  
    }  
}
```

Output

Enter the text: Hello

SHA-1 Message Digest: f7ff9e8b7bb2b91af11b495bfe5a0d8f7f3b1d5f

Result

Thus the Secure Hash Algorithm (SHA-1) has been implemented and the output has been verified successfully.

Exp No : 08	Demonstration of Intrusion Detection System (IDS) using Snor
Date :	

Aim

To demonstrate the working of an Intrusion Detection System (IDS) using the Snort tool for monitoring network traffic and detecting suspicious activities.

Procedure

Step 1: Install Snort

1. Download and install Snort.
2. Configure the network interface.
3. Verify the installation by executing:

```
snort -V
```

Step 2: Run Snort in Sniffer Mode

```
snort -v
```

Purpose: Displays packet headers on the console.

Step 3: Run Snort in Packet Logger Mode

```
snort -dev -l log
```

Purpose: Captures and stores packets in the log directory.

Step 4: Create a Simple Detection Rule

Create a file named local.rules and add the following rule:

```
alert icmp any any -> any any (msg:"ICMP Ping Detected"; sid:1000001; rev:1;)
```

Purpose: Generates an alert whenever an ICMP (Ping) packet is detected.

Step 5: Run Snort in IDS Mode

```
snort -A console -q -c /etc/snort/snort.conf
```

or in Windows:

```
snort -A console -i 1 -c C:\Snort\etc\snort.conf
```

Step 6: Generate Traffic

Open another terminal and execute:

ping google.com

or

ping 8.8.8.8

Snort detects the ICMP packets and generates alerts.

Output

Running in IDS mode

```
[**] [1:1000001:1] ICMP Ping Detected [**]  
[Priority: 0]  
06/23-10:15:30.123456  
ICMP TTL:64 TOS:0x0 ID:12345 IpLen:20 DgmLen:84  
192.168.1.10 -> 8.8.8.8  
Type:8 Code:0 ID:1 Seq:1 ECHO
```

Result

The Intrusion Detection System (IDS) was successfully demonstrated using the Snort tool. The system monitored network traffic, detected suspicious packets based on predefined rules, and generated alerts successfully.